Attila Egri-Nagy

Poetry of Programming

An elementary introduction to functional programming in Clojure

v2026.11.03

These notes are used for the MAT340 (previously MAT245) course at Akita International University in Japan and for the AS-009 course at UCU Open University for displaced Ukrainian students. The content is stable now, covering one semester. Comments are welcome! When reporting errors please specify the version number. The latest version can be downloaded from `https://egri-nagy.github.io/popbook/`

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 4.0 International" license.



As science and technology advance, their fundamental ideas get more straightforward and better explained. Therefore, more people, including younger ones, can understand how the world and our machines work. Similarly, computer programming has also become more accessible. Being a software engineer is still as back-breaking as ever, but now anyone can experience the thrills of problem-solving by instructing computers.

Here we aim to introduce programming ideas purely and minimally. We focus more on conversations with a computer and less on the tools required for software engineering. Hence the language choice: the functional core of Clojure in its interactive command line REPL (read-eval-print-loop). It is an ideal first language with a "cheeky" character: it realizes many programming concepts with ridiculously simple constructs. Accordingly, we select the features that give maximal empowerment, enabling students to solve programming challenges as soon as possible, but we do not introduce the complete language.

"THE PROCESS of preparing programs for a digital computer is especially attractive, not only because it can be economically and scientifically rewarding, but also because it can be an aesthetic experience much like composing poetry or music." – the first sentence of the *Art of Computer Programming* by Donald E. Knuth [6].

Contents

What is programming? 5

Function composition first 8

Arithmetic done with functions 12

Asking yes-or-no questions: predicates 15

Strings 17

List, the most fundamental collection 19

Vector, sequential collection that is also a function 23

Making memories 25

Defining functions 28

Manual function calls 32

Lazy sequences of numbers 34

Functional collection transformation 37

Selecting things from sequences 39

Decision making – conditionals 41

The logic of truthy and falsey 43

Reducing a collection 47

Hash-maps 50

Hash-sets 52

The sequence abstraction 54

Immutability 56

Iteration 58

Recursion 60

Destructuring 62

Point-free style 65

Sophisticated sequence construction 67

Changing the world – Side-effects 69

Etudes 71

Functional Programming 73

Index 75

What is programming?

SUMMARY: Here we attempt to give a definition of computer programming with the purpose of setting style for this book: conversations with the computer.

How to instruct computers to do something? Computers carry out instructions in an extremely precise manner. Therefore we have to specify the tasks with thorough exactness. Work done by computers has a general form: based on the input information, we would like to get some desired output. This observation makes our question more specific. *What is the most rigorous way of defining input-output pairs?* The mathematical language of *functions* is the primary form of talking to computers in a programming situation. One way or the other, all programming languages are about using, defining, transforming, and composing functions. We specify a function and give its input values, then the computer figures out the value of the function. We say that we *call* a function, and the computer *evaluates* the function with the given arguments. It is a simplifying view, but roughly this is how we play the game of programming.

The input-output pair is one particular way to look at computation. Alternatively, one can view computation as an interactive process. This approach better describes our everyday interactions with computers. It is also possible to argue that the two approaches are not different. Interaction can be understood as very short cycles of input to output computations.

Programming is also about *writing text*. This text, the *code*, is interpreted and executed by computers. However, that is not all of it. It is an often neglected aspect of programs that they are also there for humans to read. Thus, written code is also a way of communication between humans. The program I wrote will tell you how I think about solving a particular problem. The parallel between natural and programming languages can be taken further. Natural language is used not just for everyday communication but for expressing personality, emotions, and beauty. Similarly, programming languages can express ideas, ingenuity, wisdom, wit, and beauty beyond the mundane tasks of application development and maintenance. Here we aim to develop the ability to appreciate the beauty in written code. It is not just valuing a solution for its practical value but also recognizing the style expressed in the process. There is a slight difference from poetry, though. Reading the text is not enough. The joy of coding can only be experienced by doing it.

"A computational process is indeed much like a sorcerer's idea of a spirit. It cannot be seen or touched. It is not composed of matter at all. However, it is very real. It can perform intellectual work. It can answer questions. It can affect the world by disbursing money at a bank or by controlling a robot arm in a factory. The programs we use to conjure processes are like a sorcerer's spells. They are carefully composed from symbolic expressions in arcane and esoteric programming languages that prescribe the tasks we want our processes to perform." – this is the auspicious beginning of legendary MIT programming textbook, colloquially known as SICP [1].

There are three levels of understanding of procedural knowledge, i.e. knowing how to do things.

1. *Seeing someone else doing it, or listening to an explanation.* These can be entertaining but seldom lead to real learning.

2. *Doing it.* There is no need to argue that knowing the steps does not guarantee that we can carry out the process. Simply the descriptions of the actions leave so many aspects unexplained.

3. *Explaining it to someone else.* Being able to do something is not the highest form of understanding. Teachers would testify that they could understand their subject when they started teaching it.

Replace 'someone else' with 'computer' in the last item, and you will see why writing computer programs is an efficient way of understanding the world around us. Writing code requires pulling things apart and rebuilding them from primitive building blocks. Therefore computational problem solving forces us to understand the real-world problem in its details and its big picture.

Learning a programming language is like learning a foreign language, but easier and faster than that. The 'grammar', the syntax, is less complex, and 'meaning', the semantics, is aimed to be unambiguous. So learning a programming language is easy while learning a natural language is more complicated— the opposite of what people would think based on the number of people speaking foreign languages and the number of software engineers.

This book aims to give a lightning-fast introduction to the core Clojure language. By reading and rereading this text and by trying out the examples, one should gain enough knowledge to tackle programming puzzles in the style of programming contests and coding katas. The imagined reader is someone who may not become a professional software developer but does not want to miss out on this intellectual adventure (for instance, modern Liberal Arts students). People with some programming experience could benefit from the book too, but they need to maintain an open mind. Experience shows that programmers with a more traditional imperative and object-oriented coding knowledge could find it difficult to absorb the functional ideas. They may also find the exposition peculiar if not revolting. The code examples are there for maximizing the speed of worldview expansion. They may not coincide with the best practices and accepted idioms for writing efficient and readable code. We assume that the curious reader will acquire good taste by writing and reading code rather than merely reading this book.

What is the purpose and benefit of learning programming? In our culture, computing is pervasive. It is common wisdom that programming skills are helpful. On a more philosophical level, writing computer programs is about modeling and understanding some part of reality. Therefore, *learning to code teaches how to think clearly and efficiently*.

WARNING!!! This text is dense. It introduces at least one new



`http://clojure.org`
Clojure is a general-purpose, practical functional language, hosted on Java platform. For a good understanding of the design decisions involved in creating the language, the best reading is the history written by its creator [3].



`http://clojurescript.org`
A dialect, ClojureScript is hosted on JavaScript, thus it is available in browsers.

concept in each example. Reading without trying out the examples will have little effect. Copy-pasting is also discouraged since it does not really contribute to the learning process. The best strategy is to have interactive sessions as conversations with the computer. In short, the general advice is **Play in the REPL!**

Function composition first

SUMMARY: The concept of a function (from school mathematics) proves to be useful. It just needs to be generalized from numerical functions (number in, number out) to any other data transformation. Function compsition also appears here as the first example of higher order functions (whose inputs and output are functions), just to demonstrate that they are straightforward.

Programming computers can be done by using functions. But what exactly are functions? We assume at least some vague memories of functions from school, like $f(x) = 4x^2 + 2x - 4$. A function takes a value and produces another value. In this example the input value is a number, for example $x = 1$. In that case f produces the output $f(1) = 2$, since $4 \cdot 1^2 + 2 \cdot 1 - 4 = 4 + 2 - 4 = 2$. Almost all examples of functions in high school math are of this kind: they take numbers and produce numbers. This is a very limited usage of the function concept. In general, the input of a function can be anything, text or some composite piece of information; and the same is true for the output.

What is a function in general? Metaphorically, a function is a machine that can produce an output from some input value(s). We expect that a machine produces *valid outputs for all meaningful input values*. Thus, the sets of possible inputs and outputs are part of the definition of a function. We also expect that we get the *same output for a particular input all the time* (with the notable exception of probabilistic functions).

The mathematical definition of a function is just the precise description of the machine metaphor in the language of set theory and mathematical logic. Also, the set of inputs X is called the *domain*, while the set of possible outputs Y is called the *codomain*.

A function $f : X \to Y$ is a subset of the direct product (ordered pairs)

$$X \times Y = \{(x, y) \mid x \in X, y \in Y\}$$

such that

1. $\forall x \in X, \exists y \in Y$: $(x, y) \in f$,
2. $\forall x \in X, \forall y_1, y_2 \in Y$:

   $(x, y_1) \in f$ and $(x, y_2) \in f \Longrightarrow y_1 = y_2$,

i.e. for each input $x \in X$, there is at least one and at most one output $y \in Y$, giving exactly one ordered pair (x, y) in f.

It is one thing that we can ask a computer to evaluate a function already available, and it is another thing one to produce a function which is needed for our purpose. This is the difference between being a *user* and a *programmer*. Function composition is a simple way of creating new functions. Therefore, our first goal is to understand function composition as quickly as possible. We use the word *composition* in the sense of putting LEGO bricks together.

List notation for functions

For the sake of precision, we need to develop a new notation for functions, which is more suitable for computers. For a function $f(x)$, the input variable is x is also called the *argument* of the function. For the computer we write `(f x)` instead of $f(x)$. The function symbol f jumps behind the parenthesis but keeps a bit of distance from its argument by putting a space between them. There can be more than

one arguments, for example `(g x y z)` has three arguments. Even there are functions with no arguments, simply written as `(h)`. All that matters that functions are written as a *list* denoted by parentheses. The first element is the function, the following ones are arguments. By convention, we also call this list of a function and its arguments a *function call*. One can think of this as grammar rule, a minimal syntax: **functions and its input arguments are written as a list of symbols between opening and closing parens, i.e. round brackets '(' and ')'.**

A precise mathematical theory for describing functions is *lambda-calculus* [4]. Luckily, here we can proceed without learning that formalism. But it nicely explains the λ symbol in the logo.

In algebra we also write $y = f(x)$. So where is `y` in programming? That is the computer's answer for the 'question' `(f x)`. The process of coming up with an answer is *function evaluation*. We can think of the computer as a machine that has a penchant for computing functions. An opening paren triggers this habit, and the first symbol, the name of the function, determines what to do with the rest of the list. **The first element of a list is expected to be the name of a function, the rest of the list contains input data items for the function. The default behaviour is to compute the function.** So, in order to get answers for computational problems we have to construct functions whose values are the solutions. This is the essence of functional programming.

Parentheses are the symbols () put around a word or a phrase. In the Lisp family of languages these are used so often, so we got tired of using their real names. So, instead of parenthesis we say paren, and for parentheses (plural) we say parens.

The simplest function: `identity`

What is the simplest function? The function that does nothing, $f(x) = x$. Given the input the *identity function* just returns it back. In elementary mathematics we rarely talk about more than three functions, so the symbols f, g, h are often enough. In programming we define tons of functions, so naming them is quite an issue (some say the biggest). It is thus important to give functions nice names. We call the simplest function `identity`.

```
(identity 42)
42
```

These are snippets of conversations with the computer. You enter the function call (the first line), then the answer appears below. Instead of writing an essay or book, which will be read by someone else later, we have this interactive, question-answer style communication.

It works for numbers as expected. Same for text, but we have to put the sequence of letters and other symbols into double quotes. We will use a more technical term for textual data items, they are *strings*. Numbers and strings are examples of *data literals*. They are values that we write explicitly into the code or the REPL, as opposed to computing them. In other words, they evaluate to themselves. Data literals are literally just data.

```
(identity "Hello World!")
"Hello World!"
```

We give the function a traditional geek greeting in a string and it simply gives it back. No surprise, no excitement. So let's do something unusual.

```
(identity identity)
#<core$identity clojure.core$identity@1804686d>
```

Other computers may give slightly different answer, as this internal representation of functions is system-dependent.

Whoa! What's that? We asked for trouble, and we got it. Something scary came up from deep inside the system. It looks like some sort of internal representation of the `identity` function. We revealed the true identity of `identity`!

Poking the guts of the system is not our purpose here, so we will not pursue this investigation any further. However, what happened offers a remarkable insight. **Functions can take functions as arguments.** While `identity` does this only as a contrived case, we will see that there are functions designed to take other functions as arguments and return functions as well. This is a big deal, this is what makes functional programming functional.

Growing and shrinking numbers: `inc dec`

Let's pretend we don't have the numbers (except zero), so we need to construct them. In mathematics, in order to build the set of natural numbers $\mathbb{N} = \{0, 1, 2, 3, \ldots\}$, we need only two things: zero and the function $f(x) = x + 1$. Thus, $1 = f(0)$, $2 = f(f(0))$, $3 = f(f(f(0)))$, and so on. In Clojure this function is called `inc`, as an abbreviation for incrementation.

Let's say, we have a faulty keyboard. The digit keys do not work, except zero. Therefore, we do not have a way to enter numbers. Not a realistic scenario, but in foundational investigations in mathematics it is a very important tool (see Peano axioms).

```
(inc 0)
1
```

How can we do bigger numbers? Just like in math, nesting the function, using our slightly different notation.

'Nesting' in computing means that some object can contain some other object of the same kind. In other words, information is represented in a hierarchical manner.

```
(inc (inc (inc 0)))
3
```

Observe, that when a value is expected, we can simply call another function, since functions produce values. So in the general scheme of `(f x)`, the argument `x` can be replaced by something like `(g y)`. For `(f (g y))` Clojure will evaluate `(g y)` first, then the value will be fed into `f`. **Function calls can be nested.** By the way, we can also construct negative numbers by `dec`.

```
(dec 0)
-1
```

And no worries, we have all the numbers in Clojure.

Composition: `comp`

Nesting function calls has the unwanted effect of parentheses piling up. We can spare them by a simple trick. Instead of nesting the function calls, we can combine the functions first then apply the composite function to the argument. In mathematics we write

$$f \circ g(x) = f(g(x)),$$

while in Clojure

```
((comp f g) x)
```

means

```
(f (g x))
```

.

Note the order, the rightmost function in the composition gets executed first.

We can compose arbitrary number of functions.

```
((comp inc inc inc inc) 0)
4
```

If $f(x) = x + 1$, then we would write this with function composition $\circ$ in algebra as $(f \circ f \circ f \circ f)(0)$ instead of $f(f(f(f(0))))$. The calculated value is the same, but the two solutions are different, in a – let's say – ontological, metaphysical sense. In the composition case a new entity, a new function is created.

We can compose different functions, if the output of one function can be eaten by the other function. This is true for `inc` and `dec` since they expect and produce numbers.

```
((comp inc dec) 1)
1
```

Here we create a function that takes a number, decrements it, then increments the result. This is of course just a very roundabout way of saying that we want the identity function for numbers.

The general identity function can also be produced by `comp`. Given no arguments, it exactly returns that.

The importance of the identity function becomes obvious on the level of abstract algebra: it is the neutral element of functions under composition. What this means for everyday programming is that we have sensible defaults. While most of the time we want to compose two or more functions, nothing bad happens if we try that with less.

```
((comp) 19)
19
```

Consequently, `comp` works for a single argument as well. Given a function, it composes the function with the identity function, so `(comp inc)` is very much like `inc`.

Arithmetic done with functions

SUMMARY: Arithmetic calculations are fundamental computations (they were original motivators for building computers), so programming languages can be used as calculators. There is nothing surprising here, except the prefix notation, that many people complain about. However, consistency and elegance eventually pay off.

Computers, before they did anything else, performed arithmetic calculations. We do not usually think about arithmetic calculations in terms of applying functions, we just simply add, subtract, multiply and divide numbers. This is inconsistency on our side. It is better to get rid of it, so we will write arithmetic calculations as function applications.

Numbers

Numbers are data literals, so they evaluate to themselves. In other words, Clojure does not have to do any work to figure out their meanings. It just returns the value itself.

```
5
5
-5
-5
0.12
0.12
```

The basic operations: `+ - * /`

Addition is the simplest algebraic operation. Here it is in Lispy style.

```
(+ 2 3)
5
```

Very unusual after writing $2 + 3$ for many years, but there are numerous benefits. First of all, when adding more than two numbers together, we do not need to repeat the + symbol.

```
(+ 1 2 3 4)
10
```

One can mentally put the + operator between the arguments, if that helps to read the expression.

We can again make an important observation. **A function can have different number of arguments, from zero to many.** We can

depart from the idea of operators working on two arguments only. This might be a bit confusing for the asymmetric operations.

```
(- 10 1 2 3)
4
(/ 64 2 2 2)
8
```

These are $(((10-1)-2)-3) = 4$ and $(((64/2)/2)/2) = 8$. Again, the trick is to put the operators between the arguments if the expression is confusing. However, we do not have to write all the extra parens.

Surprisingly, we can do addition and multiplication with less than two arguments.

```
(+ 17)
17
(+)
0
(*)
1
```

When there is no argument, `+` and `*` are constant functions. The default values they give come from abstract algebra (additive and multiplicative identities, neutral elements). For subtraction and division both `(-)` and `(/)` complain about not having arguments. But their one argument versions do rely on the default values.

Why using some fancy piece of mathematics? They provide sensible default values. We may get a collection of numbers over the network, and our to task is to sum them up. Easy, but what happens if the collection is empty? Should the program crash, or shall we do some checking for that condition, or would zero suffice in that case?

```
(- 1)
-1
(/ 2)
1/2
```

Yes, Clojure can do rational numbers (but ClojureScript has a weaker host, so it cannot), meaning that we can have the precise symbolic value for $\frac{1}{3}$, instead of some rounded value like 0.3333333.

Being a hosted language means that inner workings of the language are written in a different language. For Clojure the host is Java, for ClojureScript it is JavaScript. This also explains why the error messages look so strange (or familiar): they come from the underlying host.

Another benefit of the function notation is that there is no need for precedence rules. What is the value of $2 + 3 \cdot 4$? Well, it depends. There are two possibilities $(2 + 3) \cdot 4 = 20$, and $2 + (3 \cdot 4) = 14$. So we have to put the parens or agree that multiplication has to be done first. In Lisp the problem is non-existent.

```
(+ 2 (* 3 4))
14
(* 4 (+ 2 3))
20
```

The price to pay: unusual notation; the gain: no ambiguity. Thus, the advice is: get used to this notation and you will like it.

People tend to complain about this prefix notation, as they might mistake unfamiliarity for difficulty.

We can also use numbers with decimal fractions, but they are 'contagious'. Once they appear in an arithmetic expression, the result turns into a decimal number as well.

```
(* 2 (+ 1 2 3.45))
12.9
(/ 0.7 2)
0.35
```

Beyond the basic operations

How about some more advanced mathematical calculations? Mathematical constants, trigonometric functions? They are also standard features of any general purpose programming languages, so Clojure simply drops down to its host language. They can be found under `Math/`.

```
Math/PI
3.141592653589793
(Math/cos 0)
1.0
```

Asking yes-or-no questions: predicates

SUMMARY: Another departure from the limited view of functions as numerical transformations only. Here we study functions that return logical values.

The simplest type of questions are where the answer is yes or no. These are called *predicates* in mathematics, and the computer says `true` and `false` instead of yes and no. Functions that return these logical values we call *predicate functions*. It is a nice habit to name those ending with a question mark.

```
(zero? 0)
true
(zero? 1)
false
(neg? -1)
true
(pos? -1)
false
```

Some predicates are so obvious that they don't even need a question mark to signal their nature. We could say `smaller?` but we got used to < in mathematics, and the latter is lot easier to type.

It's not exactly a top achievement that the computer can tell that zero is smaller than one. This is just for demonstrating the way of asking the questions. The real usage of predicates will be clear when will use symbols that can mean a wide range of values. Much like in math, $3 < 4$ is a fact, but $x < 4$ is an expression that depends on x, and divides the set of real numbers into two sets (solutions and non-solutions).

```
(< 0 1)
true
(= 1 1)
true
(= 0 1)
false
```

We can check the equality of more than two things in one go.

```
(= 1 (dec 2) (inc 0) (/ 7 7) (+ 3 -2))
true
```

Similarly, we can conveniently check whether the arguments are strictly increasing or non-decreasing.

```
(< 1 2 3 4 5 6)
true
(< 1 2 2 3)
false
(<= 1 2 3 4 5 5)
true
```

The nature of things: types

Things can be classified according to their nature, based on what they are. Forty-two is a number, `comp` is a function. In computing these classes are called *types*.

```
(number? 42)
true
(number? comp)
false
(fn? 42)
false
(fn? comp)
true
```

Clojure is strongly and dynamically typed, meaning that every data item has a well-defined type which is checked when the program is running. So, you don't need to type types (no pun intended).

There are further distinctions for numbers, roughly following the types of numbers we have in mathematics.

```
(integer? 3)
true
(rational? (/ 1 2))
true
```

However, there are some subtle differences from the mathematical classification of numbers. Numbers with decimal fractions have a different representation, bit more complicated then storing integer numbers. They are called *floating point* numbers. So 3 is the same as 3.0, but their types are different.

```
(integer? 3)
true
(float? 3)
false
(integer? 3.0)
false
(float? 3.0)
true
```

Strings

In elementary mathematics we mainly deal with functions that take numbers and produce numbers. As a departure from that, we introduce functions that work with textual information.

First terminology. In computing, letters and symbols are called *characters*, and words and sentences are called *strings*. This is not just arbitrary naming. It is act of abstraction. With a more general concept, we can cover more things: characters are not just letters, but numerical digits, punctuation marks, other signs, whitespaces and control characters. A character is a symbolic unit of information. In Clojure characters are denoted by a starting backslash.

Note that strings have parts.

```
(char? \a)
true
(char? \8)
true
(char? 8)
false
```

Strings are sequences of characters. The sequence can be empty, or just a few characters long, or a whole novel. They are denoted by double quotes.

```
(string? "tumbleweed")
true
(string? "")
true
(string? "Alice: How long is forever?")
true
```

Strings and characters are different things. This is easy to see as strings tend to have more than one character, but the edge case of a string consisting of a single character might be confusing.

In the ClojureScript dialect, the answer will be different, since the underlying language, JavaScript does not make the distinction.

```
(= \a "a")
false
```

Similarly, a number and a sequence of characters (digits) are of different *types* of data. Therefore, they are not equal even if in some sense,

for us, they represent the same quantity.

```
(= 42 "42")
false
```

This also shows a fundamental fact: anything can be represented as text, therefore as strings. We can create strings from anything by using the `str` function. It takes arbitrary number of arguments, converts each arguments into a string, then joins them into one string.

```
(str \a \b \c \space \1 2 (inc 2))
"abc 123"
```

Before we introduce other string functions, it makes sense to do a technical step to avoid typing much. For example, there is a function called `upper-case` that given a string returns another with the same letters but all capital. This function's full name is `clojure.string/upper-case`. It's rather long and it would be tedious and unreadable to type it often. After entering

```
(require '[clojure.string :as string])
```

we can use advanced string functions easily.

What happens here is that string functions live in a different *namespace*, which is like being in a folder. We just give a more convenient access to that folder. Other programming languages might say that we load a library. Organizing your software properly into namespaces, modules, libraries is a very important part of software engineering, but it will not be discussed here.

```
(string/upper-case "old pond frog leaps in water's sound")
"OLD POND FROG LEAPS IN WATER'S SOUND"
(string/capitalize "i forgot.")
"I forgot."
(string/capitalize (string/lower-case "DO NOT SHOUT!"))
"Do not shout!"
```

Much of string processing is about dealing with parts of strings (substrings).

The most advanced way of dealing with strings is using *regular expressions*, which is sort of a mini language itself.

```
(string/ends-with? "banana" "na")
true
(string/starts-with? "apple" "pp")
false
```

A powerful way of transforming strings is by replacing substrings.

```
(string/replace "Banana and mango." "an" "um")
"Bumuma umd mumgo."
```

List, the most fundamental collection

Simple values, which mathematicians would call scalars, are the atoms of data. Numbers and logical values do not have any parts, we treat them as a single unit. However, the world is more complicated than that, so the data describing everything around us comes in bigger chunks. *Data structures* are combinations of scalar data and other data structures. If the simple scalar data items are the atoms, then compound data structures are the molecules of the world of information.

At another level of abstraction we could treat scalar values as composite. For instance, an integer number has a binary representation, a sequence of zeros and ones, called bits, grouped into bytes. However, here we treat numbers atomic.

Collections are straightforward examples of compound data structures. They are 'things' as well. A bag of five apples is not the same of five apples. The bag is easier to carry.

Since we have pieces of data which contain some parts, a new question arises. *How to access the parts of data structure?* The different strategies for addressing the elements of a collection lead us to different collection types.

Creating lists

Lists are the most fundamental data structures in Lisp-like languages. They are simple a bunch of items in a sequence inside a pair of parentheses. Looks familiar? Sure! We have been using them from the beginning. Function calls are represented as a list. Given a list, Clojure will try to call the first element as a function with the remaining elements as arguments. It is so eager to evaluate, that if we just want to have a plain list of numbers, then we need to tell Clojure explicitly to stand back and not to try to evaluate the list as a function call. We have to quote the list. Otherwise, we get an error message saying that a number is not a function.

The name Lisp stands for LISt Processing.

This is the code-as-data philosophy of Lisp. The technical term is *homoiconicity*, the same-representation-ness in ancient Greek. The upshot is that programs can work on lists, and programs are written as lists, so programs can work on themselves.

```
'(1 2 3)
(1 2 3)
```

The apostrophe is just a shorthand notation. The full form of quoting looks like a function call.

```
(quote (1 2 3))
(1 2 3)
```

However, if you have the suspicion that quoting is not really a function call (since the argument is not evaluated), then you are right. It is a *special form*.

For now it is enough to know that the purpose of special forms is to do things that are not function calls.

Alternatively, we can use the `list` function that takes arbitrary number of elements and put them together in a list.

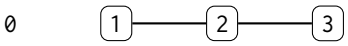
```
(list (* 6 7) (inc 100) (/ 9 3))
(42 101 3)
```

We can also construct a bigger list by combining an element and a list.

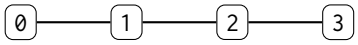
Lists prefer to connect to elements in the front. The first element is the most accessible one, for all the other we have to walk through the sequence.

```
(cons 0 (list 1 2 3))
(0 1 2 3)
```

Here we called the `cons` function with a number `0` and a list `(1 2 3)` freshly created by `list`,



and it returned a longer list containing `0` as well, attached to the beginning of the list.



There is no restriction on what sort of elements you can have in a list.

Compare lists to strings, that are sequences of characters only.

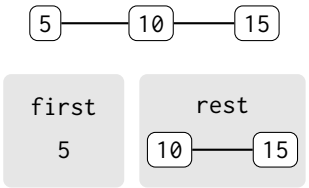
```
(cons "a string!" (list 1 2 3))
("a string!" 1 2 3)
```

The structure of lists

How about if we want to dismantle a list? That can be done along the same scheme: we separate the first element and the rest of the list.

Historically these functions `first` and `rest` were called `car` and `cdr` referring to memory locations in the IBM 704 computer, in the late 1950s. And continued to be called like that long after those machines disappeared. Sometimes it is nice to break with the tradition.

```
(first '(5 10 15))
5
(rest '(5 10 15))
(10 15)
```



The name of nothing: `nil`

Now, a tricky question. What is the first element of the empty list? Well, it's nothing. But remember, the computer needs precision, we have to be exact even when we talk about nothing. Therefore nothing, the void, the vacuum, the nonexistent, the empty, the oblivion is called `nil`.

```
(first ())
nil
```

Note that the empty list needs no quoting, since there is no danger to take the first element as a function. Only nothing is nothing, everything else is something.

```
(nil? 0)
false
(nil? ())
false
(nil? false)
false
(nil? nil)
true
```

What is the rest of the empty list? We agreed that `rest` will return a list no matter what. Also, we cannot produce a non-empty list from thin air. That leaves only one choice.

```
(rest ())
()
```

Size of a list

We can ask for the number of elements contained in a list.

```
(count '("a" \b 1.2))
3
```

There is a predicate for deciding whether a list is empty or not.

```
(empty? ())
true
(empty? '(1 2))
false
```

If `empty` was not available, how can we test for emptiness? We could use the size of the list. For example, `(zero? (count '(1 2)))`.

Concatenation

We can build lists by using other lists as building blocks, by connecting them in a given order.

```
(concat '(1 2) '(3 4))
(1 2 3 4)
(concat '(3 4) '(1 2) '(5 6))
(3 4 1 2 5 6)
```

`concat` can take arbitrary many arguments, including one and zero.

```
(concat '(\a \b c))
(\a \b \c)
(concat)
()
```

Concatenating a single list is just that list. Concatenating nothing is just another way to produce the empty list.

`reverse` *and* `last`

Here are two self-explanatory functions for lists.

```
(reverse '(1 2 3 4))
(4 3 2 1)
(last '(5 6 7))
7
```

This is a recurring theme in the learning process. Pretend that some existing function is not available, then write it.

But what if only `reverse` was available. Could we somehow get the last element of the list without `last`? The last element is the same as the first of the reversed list. We have `first`, so simple function composition works.

```
((comp first reverse) '(5 6 7))
7
```

Vector, sequential collection that is also a function

Vectors are sequences of some elements enclosed within square brackets. On the surface they very much look like lists, but with a different delimiter. We'll see later that differences between lists and vectors are fundamental. We can create a vector the short way as a data literal

Using other delimiters beyond parentheses is a great innovation of Clojure.

```
[1 2 "three"]
```

or the long way, by using the `vector` function

```
(vector 1 2 "three")
[1 2 "three"]
```

or we can turn another collection (here a list) into a vector.

```
(vec '(1 2 "three"))
[1 2 "three"]
```

Even just on the surface, vectors are great because we don't need to quote them. A vector is not a list so no one tries to evaluate it as a function call. So whenever we want to write down a sequence of elements, we'd better use vectors. The differences between lists and vectors are lot deeper than this. In a list we can access its elements one-by-one, walking through its structure by `first` and `rest`. If we want to get the last, we have to visit each element. Vectors are more accessible. They are *indexed*, meaning that each element has an associated integer number starting with `0`. For instance, the vector `["a" "b" "c"]` can be visualized as a lookup table.

0	1	2
"a"	"b"	"c"

In mathematics we say the vector is a map from $\mathbb{N}$, the set of natural numbers to a set of objects (the things we can store in a vector).

It is like a function that produces elements for index numbers. Indeed!

```
(["a" "b" "c"] 0)
"a"
(["a" "b" "c"] 2)
"c"
```

Now this is something that really widens the concept of a function! A data structure that doubles as a function. Yet another example of data-as-code. A weird puzzle:

```
((comp ["a" "b" "c"] [2 1 0]) 0)
"c"
```

Why?

There is of course a more pedestrian way to get the values from a vector. We can just ask for the *n*th element.

```
(nth ["foo" "bar"] 0)
"foo"
(nth ["foo" "bar"] 1)
"bar"
```

Accessing the elements of vectors is not as forgiving as `first`, `rest` for lists. Using an index goes beyond the elements in the lists results in an error.

Constructing a bigger vector can be done by conjoining an element.

```
(conj [11 13 17] 19)
[11 13 17 19]
```

Vectors like to connect at the end and it is easy to see why. Connecting anywhere else would mess up previous associations. Note that the order of the arguments for `conj` reflects this, just as in `cons` for lists.

Making memories

The difficulty of talking to a mathematician lies in the continuous demand for defining all the terms. 'What exactly do you mean by that?' – the oft-repeated question asks about the meaning attached to words. Same for the computer, everything has to be defined. For instance, `(+ x 1)` has no meaning unless `x` is defined. Luckily, we can attach meaning to `x`. We just call this process differently: *symbol binding*. This can be done in two ways, one permanent and one temporary.

Long term memories

With `def` we can permanently associate a name with a value. Though seemingly follows the syntax of function call, `def` is another special form.

```
(def x 2)
```

We don't display how the REPL reports successful definitions. It varies for different REPLs.

So, `x` now refers to a value, which is its meaning.



Once defined, we can use the name `x`, and it will evaluate to its bound value. Wherever we want to use `2`, we can write `x` instead. The computer will know what we are talking about.

```
(+ x 1)
3
```

In technical language, we call the words with meaning used in Clojure *symbols*. The symbols can be just single letters, or a sequence of letters mixed with numbers and some other signs like `?`,`-`,`_`, etc..

We can also tell Clojure to stop figuring out the attached meaning. If we quote a symbol, we get the symbol back.

```
'x
x
```

Where are these definitions stored? It is called the *environment*, and it is the memory where every well-formed expressions in the REPL gets its meaning from. This can be thought of as long-term memory. This metaphor gives a hint why that redefining memories is not a good idea.

Since both symbols and strings are just sequences of characters, it is very important to distinguish them. The string `"x"` is a data literal, just a piece of data that evaluates to itself. The symbol `x` is a name for something else, for example a piece of data or a function. It can also be undefined. Note that quoting symbols and the double quotes for strings are different.

```
(symbol? 'x)
true
(symbol? "x")
false
(string? 'x)
false
(string? "x")
true
```

Short term memories

Often we want to store the results of partial computations, in order to save work. Instead of

```
(+ (* 2 (+ 6 7))
   (/ 3 (+ 6 7))
   (- 2 (+ 6 7)))
198/13
```

Here the input spreads over several lines. This is no problem, Clojurewill know that you are not finished when you hit ENTER, since opening parens are not closed yet. When the expression is finished, it will acknowledge that by evaluating it.

we could do

```
(def r (+ 6 7))
(+ (* 2 r)
   (/ 3 r)
   (- 2 r))
198/13
```

computing `(+ 6 7)` only once, not three times. However, this 'pollutes the environment' by leaving `r` defined. We only want `r` to have some meaning until we evaluate the expression. `let` lets us do that.

```
(let [r (+ 6 7)]
  (+ (* 2 r)
     (/ 3 r)
     (- 2 r)))
198/13
```

This is another special form. The temporary bindings are given in a vector. There can be more bindings, but they should come in pairs: symbol first followed by a value to bind to.

The bindings are only valid in the inside `let`. If the symbol is already bound to some value in the environment, that value becomes inaccessible, the let binding 'covers' it.

```
(def x 11)

(let [x 1] (inc x))
2

x
11
```

The bindings are done in order, so we can use a binding immediately to define another one.

```
(let [x 1
      y (inc x)
      z (+ x y)]
   (str "x: " x " y: " y " z: " z))
"x: 1 y: 2 z: 3"
```

This example also illustrates an important use case for `let` bindings: when we have several pieces of partial computations we can just give them names, and then for assembling the final result of the computation we can refer to them by their names.

Defining functions

When we keep repeating the same computation only with different input data, we should write a function instead. In this view, a function is just some computation with some details left out: the inputs remain unspecified. Thus, a function is an abstract computation we can use and reuse later by specifying the inputs.

The principle of 'repeated computations should go into a function' also works the other way around. *You shall only write a function if you already have several working concrete examples of the code.* This is one of the most essential pieces of advice!

In a more traditional introduction to programming, defining functions would be the second step, right after the arithmetic operators. In Clojure we have other ways of creating functions: compositions and the associative data structures with functional behaviour. The issue of crafting functions is less urgent, but this does not diminish its significance. **We solve problems and build systems by defining functions.**

One way to think about functions is that they are great work-saving tools. Same as in algebra, we use a single letter to denote a multitude of choices of things, most notably numbers. For example, when talking about square numbers, I can mention a few: $5 \cdot 5 = 5^2 = 25$, $11 \cdot 11 = 11^2 = 121$. I can also refer to all square numbers by writing $x \cdot x = x^2$ when x is any natural number. Same happens in programming. We can do concrete calculations,

This is the big conceptual leap: abstraction. Fortunately, it is taught very early in math education, so it is a natural idea for most people. Elementary mathematics is indeed very useful for learning programming.

```
(* 2 2)
4
(* 12 12)
144
```

but soon we would get tired of typing the numbers twice. Therefore, we replace the numerical value by a symbol, for instance `x`. Thus, we get an *abstract expression* `(* x x)`, whose value depends on `x`. This `x` is a hole to be filled later. Where do we get the meaning of `x`?

The official term for a 'hole' in an abstract expression is *free variable*.

The value for `x` can come from the environment. For instance, after `(def x 2)` the expression `(* x x)` evaluates to `4`; after `(def x 12)` to `144`. This method is not recommended. First, we have to be very disciplined by using the symbols: the symbol in the definition has to match the symbol in the abstract expression. Second, we make permanent changes to the environment by attaching meanings to

Humans are really bad at administration. If we have to write the same thing at two different places, sooner or later we will write something else and get genuinely surprised by inconsistent results.

symbols. This is a recipe for disaster – imagine two abstract expressions using the same symbol, but expecting different meanings.

Local bindings are much better for providing values for abstract expressions.

```
(let [x 2] (* x x))
4
(let [x 12] (* x x))
144
```

However, we now have the input value and abstract computation mixed together. A corresponding function definition looks very much like a `let` statement: `(fn [x] (* x x))`. But there is no value bound to `x`. How come? Well, this is a function, so whoever calls the function will provide the meaning for `x`.

```
((fn [x] (* x x)) 2)
4
((fn [x] (* x x)) 12)
144
```

We separated the abstract code from the input value, making the abstract part reusable. Here we create a function, apply it to some argument(s) and throw it away, it is not bound to any symbol. Coming up with good names for functions is often mentioned as the single most difficult problem in software engineering. So, it is nice to have the option of not naming them when they are short and self-explaining.

The official term is *lambda function* for anonymous, throw-away functions. Again, see the Clojure logo.

Of course, we can make memories, where the value attached to a symbol is a function. We can make a definition `(def square (fn [x] (* x x)))`, but this comes up so often so there is a shortened way to define named functions.

```
(defn square
  [x]
  (* x x))
```

The special form `defn` reads as 'define function'. The name of the newly defined function is `square`. The following vector contains the input parameter of the function (here just a single `x`); then finally comes the *body* of the function: an abstract arithmetic expression. After this function definition, we can calculate square numbers in an elegant way.

```
(square 2)
4
(square 12)
144
```

When the `square` function is called with number `2`, the function's input parameter `x` is bound to `2`, so evaluating its body gives `4`. One can think of as substituting a concrete value into `x`. Important to note, that this assignment is temporary, just valid for the function call. So even if `x` is defined in the environment, it has no effect on the function's value.

```
(def x 3)

(square 10)
100
x
3
```

There is a *global* binding of `x` and a *local* in the function. The function has its own little environment.

The real great thing about functions, that after writing them we can forget about their details. They are indistinguishable from Clojure's own functions. Therefore, we can use this newly defined function as many times I want, no restrictions.

It is conceivable that a function does not have an input argument.

```
(defn greetings
  []
  "Hello world!")
```

There is no input to depend on to have different output values, so this is a *constant* function. `(greetings)` will always evaluate to `"Hello world!"`. Also, the function body is quite simple, just a string literal.

There can be more arguments.

```
(defn rectangle-circumference
  [a b]
  (* 2 (+ a b)))

(rectangle-circumference 3 2)
10
```

Lambda function shorthand notation

Anonymous functions save us from the burden of naming them, but we still need to do some naming. We have to give names to the inputs of the function. Luckily this can be avoided as well.

```
(#(* % %) 5)
25
```

The hashmark indicates a function literal and the `%` symbol refers to the argument of the function. If there are more arguments then numbering can be used `%` or `%1` for the first argument, `%2` for the second, and so on.

Manual function calls

SUMMARY: The seemingly technical step of unpacking a collection is the general mechanism for calling functions with arguments.

Given a bag of apples, before eating the fruits, we need to unpack them. Even if it contains several items, here the apples, a bag is one thing. Removing the container, the paper bag, is an essential step.

Removing the packaging is something we need to do in programming as well. There is a clear distinction between a function taking n arguments and another taking a list with n elements. For the latter, there is only one argument. For instance, addition expects a bunch of numbers and it would not work with a list or a vector. So, `(+ [10 11 12])` gives an error message, since the `+` function expects numbers and doesn't know what to do with a vector. The numbers are packaged in a vector, `+` cannot consume them.

The `apply` function solves the unpacking problem. It can feed the elements of a collection properly as arguments to a given function. In general,

```
(apply f [x y z])
```

evaluates to

```
(f x y z)
```

which is just calling the function `f` with arguments coming from the collection. For instance, here is how `apply` helps in calculating the sum of numbers in a vector.

```
(apply + [10 11 12])
33
```

Another example is the pair of functions `max` and `min`. They take multiple arguments and return the biggest/smallest one of them. If we call them with a single argument, they just return that.

```
(max 5)
5
```

It is straightforward to find the maximum when we have only a single thing. There is nothing else to compare to. One may not expect that, but the same happens when we call it with a collection.

```
(max [22 11 33])
[22 11 33]
```

This may not be the answer we are looking for. If we want to find the maximal element of the vector, we need to use apply.

```
(apply max [22 11 44 33])
44
```

Similarly, the function `str` converts its arguments into strings and concatenates them into one big string.

```
(str [\h \e \l \l \o])
"[\\h \\e \\l \\l \\o]"
```

Why the extra backslashes? Backslash is used to indicate characters, so if we want to have backslash as a character, we need to 'backslash' it.

The answer is correct, `str` turns the vector into a string, but it may not be what we expect. If we want to turn a sequence of characters into a string, then we need to use `apply`.

```
(apply str [\h \e \l \l \o])
"hello"
```

`apply` accepts multiple arguments, but only the last argument will be treated as a collection containing more arguments. Therefore only the last argument will be unpacked if it is a collection.

```
(apply str "Hello" " " [1 2 3] " " [\w \o \r \l \d \!])
"Hello [1 2 3] world!"
```

Why is it called `apply`? If the purpose is unpacking collections, then why choosing a name that clashes with the 'applying a function' expression? The answer is simple, because it is really function application. It is somewhat manual: `apply` takes a function and a collection of arguments and applies that function to the arguments. This process is automatically triggered by the opening paren, but we can also do it explicitly. The same argument as above, just backwards: the automatic function application `(f x y z)` can also be written as `(apply f [x y z])`.

Collection unpacking is just a special use case of `apply`, though probably the most accessible one.

Lazy sequences of numbers

SUMMARY: We capture the concept of infinity inside a computer finite resources. Laziness is the key.

In programming we often need to produce an ascending list of numbers. Therefore this task is automated. The `range` function can produce numbers starting from zero up to a limit.

```
(range 13)
(0 1 2 3 4 5 6 7 8 9 10 11 12)
```

The limit is not included in the result, so `(range 0)` is a synonym of the empty list.

When two arguments are given to `range`, they are interpreted as the start of the sequence and the end of the sequence.

```
(range 2 10)
(2 3 4 5 6 7 8 9)
```

The third argument can change the step, which defaults to one. By changing the step size to two, we can produce the even and odd single digit numbers.

```
(range 0 10 2)
(0 2 4 6 8)
(range 1 10 2)
(1 3 5 7 9)
```

As the function implements a general mechanism, `range` is not limited to integer numbers.

```
(range 1.1 4.2 0.5)
(1.1 1.6 2.1 2.6 3.1 3.6 4.1)
```

Up to this point `range` gave nothing surprising. But what happens when we give no argument at all? The start point defaults to zero, but what is the limit? Well, there is no limit, or as mathematicians would say, infinity is the limit. Without arguments, `range` returns the list of all natural numbers. The following definition is valid,

```
(def natural-numbers (range))
```

and the binding is done immediately. The computer now has $\mathbb{N}$, the infinite set of natural numbers bound to the symbol `natural-numbers`. How is this possible? How can we fit infinitely many numbers into the machine that has finite memory? Being *lazy* is the solution. Not doing anything until the last minute, unless explicitly asked. Thus the burden is on who is asking for an infinite set. The computer says: "as long as you can take them, I can produce these numbers. So, you can never catch me lying about infinity. But if you don't ask, I will not do anything." In practice, however, there are limitations. If we force the REPL to print `natural-numbers`, it will sooner or later produce some error message complaining about memory not being sufficient enough.

It is a good idea to try this. One has to know how to stop a computer program when it goes rogue.

What is the purpose then? It is useful not to set artificial limits to computations. We can do computations on demand, so we can deal with arbitrary big instances of a problem, memory permitting of course, but without doing any administration to change limits. We do not have to imagine in the beginning where exactly we will stop the process. The code becomes less specific and consequently more flexible.

Laziness is a crucial idea, its usefulness will become apparent later. For now, let's see how we can deal with an 'infinite list' in practice. We can take the first five elements.

```
(take 5 natural-numbers)
(0 1 2 3 4)
```

We can also take the second five elements by dropping the first five first (still producing a lazy infinite list).

```
(take 5 (drop 5 natural-numbers))
(5 6 7 8 9)
```

When we don't know how many elements are needed, we can make it the taking and dropping more flexible by giving a predicate.

```
(take-while neg? '(-2 -1 0 1 2))
(-2 -1)
(drop-while neg? '(-2 -1 0 1 2))
(0 1 2)
```

`take-while` collects the elements from the original sequence as long as the predicate returns `true` when applied to the elements and stops when it gets `false`; `drop-while` discards elements as long as the condition is satisfied, then returns the rest. Note that these two functions may not process the whole collection. They stop when the condition fails for the first time.

```
(take-while zero? [0 0 1 2 0])
(0 0)
```

Functional collection transformation

As a recurring activity pattern, we often need to do the same operation on the elements of a collection. As a real life example one can mention peeling oranges. We do the same thing to all oranges, we peel them. In programming we say that we *transform* the elements of a collection. In functional programming this is automated. If you want to call the same function for all elements of a collection, we don't need to bother with doing it one by one. We just *map* a function across the elements of a collection:

```
(map f [x y z])
```

will evaluate to

```
((f x) (f y) (f z))
```

The archetypal example appears in every functional programming introduction.

```
(map inc [1 2 3 4])
(2 3 4 5)
```

The result is constructed as `((inc 1) (inc 2) (inc 3) (inc 4))`.

When processing a collection, the right way of thinking is to figure out how to deal with a single element of a collection, the rest is done by `map`. This means that we need to supply a function that takes an input, namely an element of the collection. Let's say we want to turn names into greetings. We have a collection to process:

```
["Curious George" "Shimajiro" "world"]
```

We have to give the way to transform an element of this vector. We take a string and append Hello in front and an exclamation mark at the end.

```
(defn hello-message
  [s]
  (str "Hello " s "!"))
```

Now we can use `map` to transform the whole collection.

```
(map hello-message ["Curious George" "Shimajiro" "world"])
("Hello Curious George!" "Hello Shimajiro!" "Hello world!")
```

Is `map` lazy?

The name 'map' may not be the most intuitive (it will also clash with hash-maps), but what would be a better term?

It is a good practice to use anonymous functions in a `map` call. However, for beginners this is a source of trouble. It is a good idea to write and test the function separately then give it to `map`, until one gains enough confidence to write functions on the fly.

```
(take 2 (map inc (range)))
(1 2)
```

Yes. Otherwise we would not be able to talk about transforming infinite collections.

Is `map` restricted to single-argument functions? No.

```
(map + [1 2 3] [40 50 60] [700 800 900])
(741 852 963)
```

It takes the first element from each supplied collection to make the arguments of the first call of the mapped function, then the second elements, and so on. Clearly, the supplied function should be able to take the right number of arguments.

Selecting things from sequences

Selecting things from a sequential collection is nicely automated. Imagine we have a vector full of all kinds of stuff, and we need to separate the elements by their nature, by their type. The function `filter` takes a predicate (a yes-or-no question) and a sequential collection as its second argument. Then it goes through the collection, applies the predicate to all of its elements, selects those that give a yes answer, and finally returns a newly built list of selected items. We can separate the elements by separating by their types.

People coming from programming languages of the imperative style may ask 'Where is my `for`-loop?'. The short answer is that there is no need for them. Higher order functions like `map` and `filter` (and other constructs later) give the same functionality of going through a collection and doing something to the elements; with the added bonus that we don't have to fiddle with a loop variable.

```
(def v [-1 0 2 "two" 'foo 42 "42" -6])

(filter number? v)
(-1 0 2 42 -6)
(filter string? v)
("two" "42")
(filter symbol? v)
(foo)
```

Since filter also returns a collection, we can do filtering again on that to get some finer selection.

```
(filter even? (filter number? v))
(0 2 42 -6)
```

People seeing `map` and `filter` for the first time might have some problems distinguishing between the two. So it is instructive to put them side-by-side.

```
(map pos? [-2 0 1])
(false false true)
(filter pos? [-2 0 1])
(1)
```

If we map a predicate over a collection, we get a list of logical values. When filtering, based on the returned value of the predicate applied to an element, we decide whether we need to put the element into the result collection or not. The size of the result for `map` is the same

as the input collection, for `filter` it can be the same size (predicate says true for all), smaller, or even zero, when there is no positive answer.

Sometimes we want to select the elements for which the predicate returns `false`.

```
(remove zero? (filter number? v))
(-1 2 42 -6)
```

The name of the function is a bit misleading, by immutability, no elements are removed from vector `v`. A new list is created for the numbers, leaving out zero.

Decision making – conditionals

When selecting elements from a sequence, we only need to supply a predicate function that describes the desired property. Then, the individual decisions for keeping or discarding elements is made for us implicitly by `filter` and `remove`. However, in other situations we want to make decisions ourselves explicitly, independent of collection processing. We are right, without the ability of choosing between alternatives, we would only have a fancy symbolic calculator.

Having conditionals is a decisive feature, as it was discovered by Charles Babbage in 1837. He switched from the Difference Engine project to the Analytical Engine, but never finished it. This probably delayed the birth of computers by a century.

if

The simplest way of making a choice is the if-then-else form. We understand this very well as we use it in natural language. If some *condition* is true, then we have the *consequent*, otherwise we get the *alternative*. In real life, we say we choose between actions, here in a functional language we choose between two values.

Here is a function calculating the absolute value of a number.

```
(fn [a]
  (if (> a 0)
      a
      (- a)))
```

Is there a way to calculate the absolute value without `if`?

```
(fn [a]
  (max a (- a)))
```

However, we simply delegate the decision to `max`, only making the decision invisible, the implementation of `max` has the `if`.

In an `if` form not all elements are evaluated. The condition after `if` is evaluated first. If true, then the consequent is evaluated and that is the value of the `if` form (and the alternative not evaluated at all - why bother if its value is not needed?). If the condition is false the alternative is evaluated, giving the value of the form. Therefore, `if` is a special form, not a function call.

Let's say we would like to calculate $a + |b|$, where $a, b \in \mathbb{R}$. With `a` and `b` bound to numerical values and using the definition of absolute value, the straightforward, 'prosaic' way to write the expression is

```
(if (> b 0)
    (+ a b)
    (- a b))
```

as we need to choose between two cases. There is a 'poetic' way as well, in a sense that we can express a deep truth with very few words.

```
((if (> b 0) + -) a b)
```

Here we emphasize that functions are first class citizens, they are treated as any other values.

cond

For more complicated, non-binary decisions we use `cond`.

```
(defn what-is-it [x]
  (cond (number? x) "number"
        (string? x) "string"
        (fn? x) "function"
        true "no idea"))
```

The conditions are evaluated one-by-one in order. When a condition evaluates to logical true, the corresponding consequent gets evaluated and returned. If no condition matches, `nil` is returned.

Instead of `true` as the last condition, it is customary to use `:else`. This is not a special syntax, it is a keyword (used in hash-maps).

The logic of truthy and falsey

How to express more complicated conditions? Just as in everyday language, we can negate statements if we want to say the opposite,

```
(not true)
false
(not false)
true
```

connect them with `and` to indicate that they are all true,

```
(and true true true)
true
(and true false true)
false
```

and use `or` if only one of the choices needs to be true.

```
(or false false false)
false
(or false false true)
true
```

When evaluating expressions with `and` and `or`, we can be lazy again. For `and` we can quit when we find a false value, and similarly we can quit when evaluating `or` when we find a true value. Those alone can decide the value of the logical expression.

Short-circuit evaluation is the computer science term for trying to save work when making decisions.

Truthy, falsey

So far so good, but let's now discuss a peculiar language feature. In Clojure we are more generous about how the logical values true and false can be expressed. **Both `false` and `nil` are treated as false, every other value is treated as true.** That is, whenever a logical value is expected, we can put any other value there. This can be a blessing, when we know about it, or a curse, if we don't. For example we can use any value as a condition in an `if` statement.

People coming from languages with a strict type system may find this hard going.

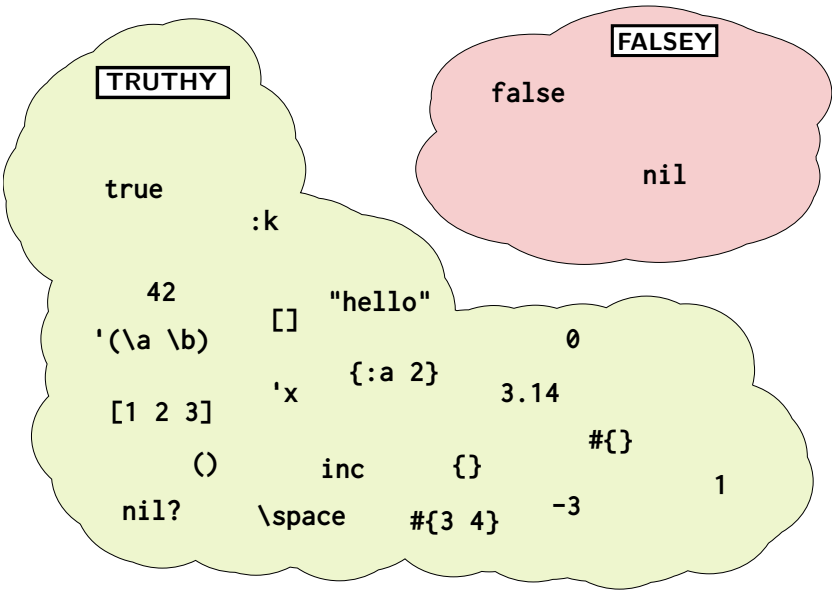


Figure 1: Falsey is `nil` and `false`, truthy is everything else.

```
(if true "consequent" "alternative")
"consequent"
(if 0 "consequent" "alternative")
"consequent"
(if () "consequent" "alternative")
"consequent"
(if [] "consequent" "alternative")
"consequent"
(if + "consequent" "alternative")
"consequent"
(if nil "consequent" "alternative")
"alternative"
(if false "consequent" "alternative")
"alternative"
```

So logical expressions can be *truthy* or *falsey*. Note this does not turn `nil` into `false`, or `1` into `true`,

```
(true? 1)
false
(true? true)
true
(false? nil)
false
(false? false)
true
```

but in decision making situations we are quite liberal about what counts as truth. What are the benefits? The flexibility in expressing selections. For instance, we can easily filter out `nil`s and `false`s from a collection.

```
(filter identity [1 2 nil \c "hello" nil false])
(1 2 \c "hello")
```

Truthiness is also good for the efficient evaluation of logical expressions. The special form `and` evaluates its arguments until it find a falsey one,

```
user=> (and (< 4 5) false (range))
false
user=> (and (= 1 (inc 0)) nil (range))
nil
```

note that the dangerous `(range)` is not evaluated at all; or if they are all truthy it returns the last.

```
(and (< 4 5) [] [1 2])
[1 2]
```

The returned value may feel strange, but an `if` statement would interpret this as a yes. The form `or` does the opposite, it returns the first truthy, or the last falsey if they are all falsey.

```
(or nil false 1)
1
user=> (or false  nil)
nil
```

Negation also accepts truthy and falsey values, and produces genuine `true/false` answers.

```
(not nil)
true
(not "hello")
false
```

Whenever you are in doubt, whether a value is truthy of falsey, you can coerce it to an honest logical value, which we call in computer science boolean.

```
(boolean nil)
false
(boolean false)
false
(boolean [])
true
(boolean 0)
true
```

Reducing a collection

Often we need to make a single value out of a collection. Or another collection, for that matter. Reducing is a very general operation, therefore it may be perplexing first. A visual image may help to get the idea. Imagine a child collecting pebbles on the seashore. Picking up stones with the right hand, while holding the already collected ones in the left palm, or in a small bucket hold in the left hand. Or the child can decide not to gather all pebbles, just the most beautiful one. For each newly picked up pebble, she compares it with the one held in the left hand, and decides which one to keep. In both cases, some result is accumulated in the left hand.

`map`, `filter` and `reduce` form an expressive set of collection processing functions. All we need to do is to supply a function that can deal with a single element of a collection, the rest is automated.

Let's see how to collect pebbles in Clojure, or rather keywords into vectors.

```
(reduce conj [1 2]  [3 4 5])
[1 2 3 4 5]
```

Here we have an initial collection `[1 2]` (the pebbles already in the bucket), and the numbers in a vector `[3 4 5]` to be collected (the pebbles still on the beach). 'Putting a pebble into the bucket' is done by `conj`ing them one by one to the existing collection. The fact that we only see the result may hinder understanding the reduction process. Luckily, it is possible to see the accumulation step-wise.

```
(reductions conj [1 2]  [3 4 5])
([1 2] [1 2 3] [1 2 3 4] [1 2 3 4 5])
```

Reducing some elements into a new collection is such a common operator that there is a dedicated function for that. Roughly speaking `into` is just `reduce conj`.

```
(into [1 2]  [3 4 5])
[1 2 3 4 5]
```

Now for finding the most beautiful pebble. This is the same type of problem as finding a maximal number from a collection of numbers.

```
(reduce max [1 2 1 3 2 5 4 6 1 2])
6
(reductions max [1 2 1 3 2 5 4 6 1 2])
(1 2 2 3 3 5 5 6 6 6)
```

Here we didn't specify the initial value of the reduction, the first value of the vector can serve as the initial value.

At this point it might be difficult to see the difference between `apply` and `reduce`, since in special cases they produce the same result.

```
(reduce + [1 2 3 4 5 6 7 8 9])
45
(apply + [1 2 3 4 5 6 7 8 9])
45
```

The + function with multiple arguments will do reductions internally anyway, so ultimately they realize the same process.

However, the shape of the process is different: `apply` yields `(+ 1 2 3 4 5 6 7 8 9)` while `reduce` will go through steps `(+ 1 2) (+ 3 3) (+ 6 4) (+ 10 5) (+ 15 6) (+ 21 7) (+ 28 8) (+ 36 9)`. What happens is that reduction requires a function with 2 arguments, the so called *reducing function*. Its first argument is an *accumulator*, some data in which we collect the result of the computation. The second is an element from the collection being processed by `reduce`. Here is a simple example of a reducing lambda function: `(fn [c _] (inc c))`. This ignores its second argument, but whenever it gets a new one, it increments the counter. This can be used for re-implementing `count`.

```
(defn my-count
  [coll]
  (reduce (fn [c _] (inc c))
          0
          coll))
```

The _ symbol is used for an argument that is not used in the body of the function. It is just a convention, the underscore is a symbol as good as any other. This emphasises that we don't need to process the elements when we just want to count them. Here is another example, where the collection is just used to provide the number of steps.

```
(reduce
  (fn [v x]
    (conj v (+ (last v)
               (last (butlast v)))))
  [0 1]
  (range 15))
[0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987]
```

These are the first 17 of the so called Fibonacci numbers.

The general form of `reduce` is this.

```
(reduce (fn [result-so-far next-item]
          (compute-new-result result-so-far next-item))
        initial-value
        collection)
```

Depending on the body of the reducing function (how the updated result is calculated by `compute-new-result`), the behaviour can be really versatile. The initial value is optional. When missing, the first element of the collection is taken to be the initial value, and the second element serves as the first `next-item`.

We can easily imagine the shape of the process of a `reduce` call. For instance,

`(reduce rf initial-value [x y z])`

turns into

`(rf (rf (rf initial-value x) y) z)`,

since `reduce` takes the responsibility of calling the reducing function `rf` for the elements of the given collection, as well as setting up the very first call.

Hash-maps

Association is a mental connection between things and ideas. The most common example is naming. We associate meaning to words, that are sequences of letters. When we have several associations at the same time, like several word, then we need a dictionary. These are called hash-maps in Clojure.

Another way to understand hash-maps is that after introducing vectors, we should do a powerful abstraction. A vector associates things, some *values* to numbers $0, 1, 2, 3, \ldots$, to the so-called *indices*. We have index-value type pairings. This is useful for some things in an ordered sequence, but it is a very special association. What if the indices can be anything? What if we could map any value to any other? That's what *hash-maps* do. The name is unfortunate, since it describes how these associations are implemented, and not what they do. Lookup table might be a better name, but we are stuck with the old name. Still about terminology, we call the generalized indices *keys*.

Curly braces indicate a hash-map, and we simply write the key-value pairs inside.

```
{"answer" 42 "question" "?" 2 "two" "list" '(1 2) () '(3) }
{"answer" 42, "question "?", 2 "two", "list" (1 2), () (3)}
```

The computer politely puts commas after the each key-value pair. We can do that as well, but it is not necessary. We associate the string `"answer"` with the number `42`, while the question itself is unknown, so the key `"question"` gets associated with `"?"`. Then number `2` is connected to its English name, the word `"list"` is associated with a two-element list of numbers, then the empty list with a one-element list. The order of the key-value pairs is not guaranteed.

Yes, some of these associations are from an old science-fiction book.

`"answer"`	`()`	`2`	`"question"`	`"list"`
`42`	`(3)`	`"two"`	`"?"`	`(1 2)`

So anything can be used as keys and values. Even nothing is permissible as a key: `{nil "nothing"}` is a permissible map. But there is a type of keys, that is specially designed for hash-maps: *keywords*.

They are symbols starting with a colon and they evaluate to themselves.

So, what was the book we mentioned?

```
(def book {:author "Douglas Adams"
           :title "Hitch-hiker's guide to the galaxy"
           :year 1979})
```

This is an all-time favorite science-fiction novel with unusually big computers playing central roles. [2]

A hash-map behaves like a small database. Here we store information about a book. You can look up values by giving keywords.

```
(book :author)
"Douglas Adams"
```

A database as a function?!? Yes, hash-maps can be used as functions with a key as the single argument. The result of the function call is the corresponding value, or `nil` if it is not in the map. There are more neat surprises.

```
(:author book)
"Douglas Adams"
```

Keywords also behave as functions! They look themselves up in a hash-map.

There is a manual way to look up a data item.

```
(get book :title)
"Hitch-hiker's guide to the galaxy"
(get book :price)
nil
```

Why is this useful? We can give a default value in case the key is not in the hash-map.

```
(get book :title "no information")
"Hitch-hiker's guide to the galaxy"
(get book :price "no information")
"no information"
```

There is an elegant way to create a hash-map when we have the keys and values in separate sequential collections, but matched up nicely. We can just 'zip' them together.

```
(zipmap (range 4) ["zero" "one" "two" "three"])
{0 "zero", 1 "one", 2 "two", 3 "three"}
```

There is also easy access to the keys and values separately.

```
(keys {:x 1 :y 11 :z 111})
(:x :y :z)
(vals {:x 1 :y 11 :z 111})
(1 11 111)
```

Hash-sets

Imagine a hash-map where keys are mapped to themselves. This doesn't make too much sense at first sight. If I have the element, why would I want to look it up in a hash-map? If the key is there, I get back the key, otherwise `nil`. These are truthy and falsey values, so a hash-set behaves as a predicate for function for deciding membership. Also, in a hash-map, keys cannot be duplicated (otherwise the output value of the lookup function would be ambiguous). These two give the behaviour of a *set* in the mathematical sense.

Hash-sets can be defined as data literals by curly braces prefixed by a `#`.

```
(def numbers #{1 2 3})
```

As in mathematical sets, the order of the elements does not matter, and it is not guaranteed to retain the order of the definition.

```
numbers
#{1 3 2}
```

As with hash-maps, we can use `get` to do lookup and to have the possibility for a default answer in case the key is not in the hash-set and don't want to have `nil` as an answer, which is falsey.

This is a bit weird, it's like asking "What's the weather like tomorrow? If you don't know, just say sunny.". But often, handling `nil` would be quite a hassle.

```
(get numbers 2)
2
(get numbers 4)
nil
(get numbers 4 :not-an-element)
:not-an-element
```

Hash-sets are also functions of their keys.

```
(numbers 4)
nil
(numbers 3)
3
```

In case more civilized logic values are needed, `contains?` is a predicate function that decides whether a hash-set contains the given key or not.

```
(contains? numbers 6)
false
(contains? numbers 1)
true
```

Thanks to truthiness, sets can be used as predicate functions in `filter`.

```
(def fruits #{:apple :mango :pear :grape :banana :orange})
(def vegetables #{:cucumber :tomato :carrot :onion})
(def basket [:apple :apple :onion :carrot :grape])
(filter vegetables basket)
(:onion :carrot)
(filter fruits basket)
(:apple :apple :grape)
```

For creating hash-sets, we can directly supply the elements to `hash-set`,

```
(hash-set 3 2 1 2 3 1 1)
#{1 3 2}
(hash-set)
#{}
(hash-set "hello")
#{"hello"}
```

or we can turn a collection in a hash-set.

```
(set [13 17 17 19 23 19])
#{13 17 23 19}
(set (range 5))
#{0 1 4 3 2}
```

It is important to remember, that sets do not make any promise about keeping the order. All they do is just answering whether a key is contained or not. However, one can have sorted sets as well.

```
(sorted-set 13 17 17 19 23 19)
#{13 17 19 23}
```

The sequence abstraction

All the collections so far, namely lists, vectors, strings, hash-maps, hash-sets can be handled as sequences of elements. We can use the `seq` function to investigate how the collections are turned into logical sequences. Being a sequence is not a surprising property for lists and vectors, since they are sequential data structures anyway.

```
(seq '(1 2 3 4))
(1 2 3 4)
(seq [1 2 3 4])
(1 2 3 4)
```

Strings are also sequences, whose elements are characters.

```
(seq "hello")
(\h \e \l \l \o)
```

For hash-sets, the elements can be enumerated one-by-one, but it is important to note that there is no promise about the order of elements.

```
(seq #{:a :b :c})
(:c :b :a)
(seq #{1 2 3 5 8 13})
(1 13 3 2 5 8)
```

How about hash-maps? They don't really have single elements in them, rather they contain pairs of elements. These key-value pairs van be represented by vectors. Accordingly, the sequence of a hash-map consists of key-value pairs in vectors.

```
(seq {:x 1, :s "foo", :c \x})
([:x 1] [:s "foo"] [:c \x])
```

So it seems that the right way to think about hash-maps that it is a collection that contains associations, treated as single things.

```
(count {:a 11, :b 13})
2
```

This also shows that there is no need to call `seq` explicitly. Clojure takes care of turning the collections into sequences, whenever needed. This way, all the sequence functions like `first`, `rest`, `first`, `second`, `last` and the higher order sequence processing function like `map`, `filter`, etc. can work with all the above data structures.

What is the point? The sequence abstraction can be justified in different ways. From the students' perspective it is a big win, since one only needs to remember functions dealing with sequences, then working with other data structures can be done without learning anything new. The software engineer relishes the fact the same piece of code would work in many different situations, enabling *code reuse*.

Here the unmissable quote is: "It is better to have 100 functions operate on one data structure than to have 10 functions operate on 10 data structures.", by Alan J. Perlis in the foreword of SICP [1].

Where is the catch? The behaviour of the same function can be different for different data structures. For example, the difference between `cons` and `conj` can be confusing. `cons` builds a sequence, always putting a new element in the front,

```
(cons 1 '(2 3))
(1 2 3)
(cons 1 [2 3])
(1 2 3)
```

while `conj` *conjoins* a new element in a way that is 'natural' for the collection, (front for lists, back for vectors)

```
(conj '(2 3) 1)
(1 2 3)
(conj [2 3] 1)
[2 3 1]
```

and it can take arbitrary many arguments.

```
(conj [2 3] 4 5 6)
[2 3 4 5 6]
```

Conjoining keeps the type of the collection.

```
(conj {:name "Arthur"} [:age 42])
{:name "Arthur", :age 42}
```

Beyond the different behaviours, performance can also be an issue. One can use `nth` on lists, but it will not be as efficient as on vectors, since Clojure has to hop through all the preceding elements.

Immutability

Consider the following little conversation with the computer.

```
(def v [1 2 3])
v
(conj v 4)
[1 2 3 4]
```

Now the question is what is the current value of `v`? What does `v` evaluate to? If your answer is `[1 2 3]` (the correct one), then there is nothing to worry about. We associated the vector `[1 2 3]` with the symbol `v`. Then, we called the `conj` function with arguments `v` and `4`, which returned the conjoined vector `[1 2 3 4]` – as a new value. The association of `v` to `[1 2 3]` has not changed.

However, if you say `v` is `[1 2 3 4]`, then you must have some previous programming experience in a language with mutable data structures. Then, CLOJURE's *immutable* data structures can look a bit strange in the beginning. In CLOJURE, you cannot change a collection. You can only create a new one. In more traditional languages, you can extend a collection, or remove elements from it. Two different worldviews. Both approaches have advantages and disadvantages. However, if we run programs concurrently or in parallel, then immutable data structures have a clear edge. We do not need to worry about someone else changing a collection of data halfway through while we are looking at it. No need to lock down any piece of data.

So, what is the immutability exactly? Simple data items never change. 42 is 42 forever. I can add one to it, `(inc 42)` equals 43. Function create new values but do not touch their arguments. Naturally, 42 will not mutate into 43 just by applying the function $f(x) = x + 1$ to it. This is so obvious, that it feels a bit silly to mention it.

For composite data items immutability is less obvious. It makes perfect sense to add a new element to a vector, by changing it to a bigger one. In CLOJURE a new vector is created and the old one is retained. Do we need to worry about memory consumption? No, it's

This is exactly how version control systems work, e.g. GIT. They store only the difference between different versions of the documents.

better than that. It is also efficient: there is structural sharing between the old and the new vector. There is no unnecessary duplication of the data items stored in vectors, hash-maps.

The naming of some built-in functions can be misleading. For instance, `update` for associative collections.

Iteration

We often need to call a function *iteratively*, i.e. feeding the output back into the function. Let's consider a simple mathematical game. Given a positive integer, if it is even then half it (still a whole number), when it's odd multiply by 3 and add 1. What happens if you do the same with the result? Again and again. The function itself is easy to write.

```
(defn c [n]
  (if (even? n)
    (/ n 2)
    (inc (* 3 n))))
```

So we can try.

```
(c 1)
4
(c 4)
2
(c 2)
1
```

From 1 we get back to 1. From 8 we would go to 4, then again get back to 1. Is this the case for other numbers? Interesting problem but tedious to call the function again and again. So, let's automate! In algebraic notation, we want the sequence

$$n, c(n), c(c(n)), c(c(c(n))), \ldots$$

we want to repeat the function c, calling it with the newly produced value. The function `iterate` does that exactly:

`(iterate f x)`

produces the lazy list

`( (f x) (f (f x)) (f (f (f x))) ... )`, so in particular we can use the above function `c`

```
(iterate c 1)
```

In mathematics this is called the Collatz conjecture, and it is a particularly tough problem. We do not know at the time being whether for all numbers the process eventually returns to 1 or not.

But when you hit enter, no result. Seemingly. The computer evaluates `c` with argument `1`, then takes the result as an argument of `c`, infinitely many times (at least until the computer's memory fills up). That's why it never returns a value. This is another example of dealing with something potentially infinite. The functions `take` and `drop` can tame infinite sequences of numbers, and they can do that to infinite sequence of function calls. We simply ask to return the first few values of the infinite sequence.

```
(take 20 (iterate c 9))
(9 28 14 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1)
```

We can also ask what is the sixth element of the sequence (indexed by 5)?

```
(first (drop 5 (iterate c 9)))
11
```

The `range` function called without arguments produces the infinite lazy list of natural numbers. Can we produce the same list without `range`? Yes, `(iterate inc 0)`.

Recursion

The simplest way of defining *recursion* is that a function calls itself. This may look paradoxical, since we are defining something in terms of itself, thus we have a circular definition. Schematically, a recursive function definition looks like `(defn f [x] ... (f ...) ...)`, where the dots stand for some other code. The point is that `f` appears in its own definition. How can we use a function we do not have yet? The paradox disappears if we consider the function calling itself with different arguments, that somehow represent a smaller and easier to solve version of the problem. So, recursion is a way of dividing a difficult problem into easier ones.

Self-reference is where things get interesting. Consider the sentence "This sentence is false."; it does not have a well-defined logical value. Beyond logical paradoxes, self-reference is often thought to be a key ingredient to consciousness. See for example [5]. Also, the movie *Inception*, dream inside a dream, is another illustrative example of a recursive structure.

When a problem is somehow given as a list we can naturally divide it into two parts: dealing with the first element, and dealing with the rest of the list. Recursive functions on lists are the classical examples of functional programming. Let's imagine `count`, a function for getting the size of a collection, is not available, so here is an implementation for lists.

```
(defn my-count
  [l]
  (if (empty? l)
      0
      (inc (my-count (rest l)))))
```

If the list is empty, then the function returns `0`. This is the *base case*, where the recursion stops. Otherwise, it knocks off the first element of the list and calls itself for the rest of the list. This way it calculates the size of the list by adding one to the size of the list without the first element. It is instructive to think about what happens when `my-count` is called.

```
(my-count [6 7])
(inc (my-count '(7)))
(inc (inc (my-count '())))
(inc (inc 0))
(inc 1)
2
```

It requires quite a cognitive effort to see the shape of this process as one has to imagine the chain of these 'hanging calls'.

Another classic is the re-implementation of `map`, again dividing the work by `first` and `rest`.

```
(defn my-map
  [f coll]
  (if (empty? coll)
    ()
    (cons (f (first coll))
          (my-map f (rest coll)))))
```

The functions `filter` and `remove` can be implemented recursively in a similar way.

Dividing the work is not limited to list operations. Here is a classic (literally, from ancient Greece) algorithm for finding the greatest common divisor of of two positive integers.

```
(defn gcd
  [m n]
  (if (zero? n)
      m
      (gcd n (mod m n))))
```

The difficulty of recursion comes from its all-or-nothing nature. The useful strategy of crafting a piece of computation in the REPL, identifying its moving parts, then turning it into a function definition, does not work. The function has to be there already. The above examples are all very simple. They in fact can be done in a non-recursive way (e.g., by reducing).

Here is a more involved recursion. How many ways can we express a particular sum value with a given type of coins (assuming infinite supply)?

```
(defn coin-change [coins sum]
  (cond
    (zero? sum) 1
    (or (neg? sum) (empty? coins)) 0
    :else (+ (coin-change coins (- sum (first coins)))
             (coin-change (rest coins) sum))))
```

For instance, `(coin-change [1 3] 7)` will return 3, since we can have seven 1-coins, four 1-coins and a single 3-coin, or two 3-coins with a single 1-coin.

Recursive solution are pretty, but not the most efficient way. Is the recursive call the last thing we do? In that case we do not need all the hanging calls. We can de non-recursion in a recursive style with `loop` and `recur`.

Destructuring

Often there is a need for extracting information from a collection. This is usually done by a systematic naming of elements in the collection. In other words, we are matching a pattern on a composite data structure.

How often do we get some collection as an argument of a function and then spend half of the body of the function to pull the collection apart? *Destructuring* handles this. Just like kicking the ball on the volley, we can give names to parts of the arguments before they land in the function.

In order to show its usefulness, we will rewrite an example several times to improve it. For example, we can represent a line defined by a vector of two points. The points are also vectors of the coordinates, a pair of two numbers. Imagine that we would like to compute the slope of the line like `(slope [[1 2] [2 4]])`. It is calculated by finding the ratio of the change in the y-coordinate and the change in the x-coordinate. Expressed algebraically,

$$m = \frac{y_1 - y_2}{x_1 - x_2},$$

so we define the function to compute this formula.

```
(defn slope
  [line]
  (/
    (- (second (second line)) (second (first line)))
    (- (first (second line)) (first (first line)))))
```

Here the coordinate information is extracted on demand, making the actual calculation obscure, littered with the retrieval. The formula is not easily recognizable in the code. We can separate the technical bit (extracting data from a collections) from the actual computation, by doing symbol bindings for each data item we need in the formula.

```
(defn slope2
  [line]
  (let [p1 (first line)
        p2 (second line)
        x1 (first p1)
        y1 (second p1)
        x2 (first p2)
        y2 (second p2)]
    (/ (- y1 y2) (- x1 x2))))
```

Now the computation is quite clear, it is basically the mathematical formula. However, we have a long list of bindings. We paid for the clarity by making the code longer. Moreover, writing the `let` statement is a boring task, so we want to automate it. Destructuring does exactly that automation of naming. We can simply give the 'shape' of the input data in the argument list. We don't just give a single name for the argument, but several names put together in the required data format.

```
(defn slope3
  [ [[x1 y1] [x2 y2]] ]
(/ (- y1 y2) (- x1 x2)))
```

`slope3` still has a single argument, but its internal structure is specified, and the binding is done by matching the names with actual data items in the input collection. This may look like magic first, but it is actually just a simple automation. It is easy to reveal how it is done.

`destructure` is very useful in figuring out what goes wrong in an unsuccessful and complex destructuring attempt.

```
(destructure '[ [x y] [13 19]])
[vec__1246 [13 19]
 x (clojure.core/nth vec__1246 0 nil)
 y (clojure.core/nth vec__1246 1 nil)]
```

It does exactly the work we did not want to do manually. `destructure` produces a vector of bindings, that is given to `let` in a real destructuring situation.

Destructuring works for all symbol bindings. It can be used in `let` statements as well.

Pattern matching on a sequence is called *sequential destructuring*. The general idea works for associative collections as well, so we also have *associative destructuring*. The matched pattern will determine the bindings.

```
(def m {:a 3 :b 5})
(let [{a :a, b :b} m]
  [a b])
[3 5]
```

The destructuring reads as the symbol `a` will be bound to the value of key `:a` in the hash-map `m`. Sequential and associative destructuring can be mixed.

In short, destructuring allows us to name items in a collection by their positions. In a sense it is a 'visual' way of systematic symbol binding.

Point-free style

We can create new functions by specifying what they do with their arguments, but we can also make new functions without mentioning those input values (points). There are several ways to do this:

- composing functions by `comp`,
- preloading arguments by `partial`,
- grouping functions to work on same input by `juxt`,
- negating logical output value by `complement`.

Nested function calls of the form `(f (g (h x)))` can be replaced by `((comp f g h) x)`.

```
(def negative-product (comp - *))
(negative-product 2 3)
-6
```

With `partial` we can create functions by preloading the first few arguments of a function with several inputs. In a sense we turn a general function into a more specific one.

```
(def ten-times (partial * 10))
(map ten-times [1 2 3])
(10 20 30)
```

When we want to apply several functions to same input, we can juxtapose them. `(juxt f g h)` is the function that produces `[(f x) (g x) (h x)]` when applied to `x`. It works functions that can take more arguments, but obviously they need to have the same number of inputs,

```
((juxt take drop) 3 [1 2 3 4 5 6])
[(1 2 3) (4 5 6)]
```

and `juxt` always returns the result in a vector.

One thing where `juxt` really shines is retrieving data items from a hash-map, more than one at the same time.

```
(def book {:author "Terry Pratchett"
           :title  "Hogfather"
           :year   1996
           :series "Discworld"})
((juxt :title :year) book)
["Hogfather" 1996]
```

Combining these higher-order functions can be very expressive. Here is a one-liner function for segregating even and odd numbers.

```
(def sgr (juxt (partial filter even?) (partial filter odd?)))
(sgr (range 9))
[(0 2 4 6 8) (1 3 5 7)]
```

It's amazing how far one can get with point-free style. Here we define a function that counts the number of lowercase letters in a string.

Of course, the standard solution for this problem would use regular expressions.

```
(def count-if (comp count filter))
(def letters (set "abcdefghijklmnopqrstuvwxyz"))
(def count-lowercase (partial count-if letters))
(count-lowercase "aBbC10x")
3
```

It is debated how desirable is the point-free style in practice. Some say it is 'pointless', and readability can be a subjective issue.

Sophisticated sequence construction

Imagine that we need to produce a list of ordered pairs of elements coming from two collections. It is easy to see that this might be done with nested `map` calls.

Mathematicians call this the *direct product*, or *Cartesian product*.

```
(mapcat
  (fn [v] (map (partial conj v) coll2))
  (map vector coll1))
```

Easy? Readable? Even if `map` is your best friend, this piece of code still requires a bit of time to read. It is a simple task, but the solution already has three `map` calls. How about the direct product of three or more sets? There has to be a better way.

This type of construction is an example of a general operation, called *list comprehension*, and it is also automated.

```
(for [x [1 2 3]
      y [:a :b]]
  (list x y))
((1 :a) (1 :b) (2 :a) (2 :b) (3 :a) (3 :b))
```

The construction is somewhat similar to `let` bindings, but the symbols are not bound to the collections, but rather to their elements one by one systematically. Then we give an expression containing these symbols, which defines an element added to the resulting sequence.

The order of the bindings does matter. Compare this with the above code block.

```
(for [y [:a :b]
      x [1 2 3]]
  [x y])
([1 :a] [2 :a] [3 :a] [1 :b] [2 :b] [3 :b])
```

The first binding is the slowest to change.

It is possible to restrict the elements in the result, much like the functionality of `filter` and `take-while`. We can produce pairs of numbers that are in strictly increasing order. With `:when` we can specify a predicate, here the less-than relation, and we collect only those items that satisfy the predicate.

```
(for [x (range 4)
      y (range 4)
      :when (< x y)]
  [x y])
([0 1] [0 2] [0 3] [1 2] [1 3] [2 3])
```

The `:while` modifier behaves like `take-while`. It adds elements to the final sequence as long as the predicate is satisfied, and then stops the binding process. Using the `:while` modifier is more tricky as it depends on the order of the enumeration and it is also sensitive to the location.

That is 'breaking the loop'.

```
(for [x (range 3) :while (not= x 1)
      y (range 3)]
  [x y])
([0 0] [0 1] [0 2])
```

Here we check the condition before going through the `y` bindings.

```
(for [x (range 3)
      y (range 3) :while (not= x 1)]
  [x y])
([0 0] [0 1] [0 2] [2 0] [2 1] [2 2])
```

Here we descend into enumerating the second level, then check the condition.

The `:let` modifier allows us to make symbol bindings in addition to the enumerated ones.

```
(for [x (range 3)
      y (range 3)
      :let [xy (* x y)]]
  (str x "*" y "=" xy))
("0*0=0" "0*1=0" "0*2=0" "1*0=0" "1*1=1" "1*2=2" "2*0=0"
 "2*1=2" "2*2=4")
```

List comprehension is like a mini-language inside Clojure with the special purpose of constructing sequences.

The industry term is *DSL*, *domain-specific language*.

Changing the world – Side-effects

The functions we have seen so far were *pure* functions: the output depends only on the inputs. In order to understand what the function does, we don't need to check anything else. The history of previous function calls have no effect on the current call. This makes things very easy. Of course, the world is not like this. If you ask me now and tomorrow 'How are you?', you are not guaranteed to have the same answer. Similarly, we have 'functions' that are not pure.

```
(rand)
0.7811104824887901
(rand)
0.3589579594682847
(rand)
0.629147957095869
```

These are random numbers between `0` and `1`. The function takes no arguments but it doesn't return the same value for each call. Something must be changing in the background. The function has a *side-effect*.

Rather, they are pseudo-random numbers generated by an algorithm.

There are cases where are only interested in the side-effect, not in the returned value.

Playing sounds, displaying graphics, motion picture, saving files, and so on.

```
(println "Hello world!")
Hello world!
nil
```

The return value is `nil`, the side-effect is the text appearing on the screen. We could postpone thinking about side-effects since the REPL automatically prints return values of functions.

Calling a function with side-effect may not be a one off case. Similar to list comprehension with `for`, we can execute repeated tasks using `doseq`.

```
(doseq [a [1 2]
        b [:x :y]]
   (println a b))
1 :x
1 :y
2 :x
2 :y
nil
```

Etudes

One can learn the rules of chess or Go immediately. However, mastering them takes long time, playing many games. Same for programming. Learning the basic constructs of a programming language is a quick process. Understanding what `reduce` does is not that hard, but knowing when to use it and how to adapt the general mechanism to a particular problem require lot of practice. Like performing musicians, we need to go through exercises several times.

What is 42?

The task is simple. Write some code that somehow produces `42`. The easiest way is to use a data literal `42`. How about other ways? When you are not allowed to write down the number itself. By arithmetic operations,

```
(* 6 7)
(+ 40 2)
```

or by incrementing/decrementing,

```
(inc 41)
(dec 43)
```

or by counting a number of elements in a collection,

```
(count "01234567890123456789012345678901234567890")
```

the ASCII code of `*` is also `42`,

```
(int \*)
```

n!

Writing a function to calculate $n!$ (n factorial) is a common programming exercise for recursion. It is defined as the product of natural numbers from `1` up to n, with the special case of $0! = 1$. Recursively,

```
(defn factorial [n]
  (if (zero? n)
      1
      (* n (f (dec n)))))
```

but if someone is worried about the inefficiency of the recursive call then tail recursion is also possible.

```
(defn factorial
  ([n] (f n 1))
  ([n r] (if (zero? n)
             r
             (recur (dec n) (* n r)))))
```

The current value to be multiplied with is also passed on through the call, so the caller don't have to wait. However, there is no real need to be recursive, thus checking for the base case is not necessary.

```
(defn factorial [n]
  (reduce * (range 1 (inc n))))
```

Transposing a matrix

Considering matrix

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix},$$

we would like to calculate its transpose (swapping rows with columns)

$$A^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}.$$

Matrices can be represented as nested vectors.

```
(def A [[1 2] [3 4] [5 6]])
```

Without further ado, here is how to do the transpose.

```
(apply map vector A)
([1 3 5] [2 4 6])
```

If the result needs to be a vector as well, then `mapv` can replace `map`.

Functional Programming

What is functional programming? The computer is doing useful work by evaluating functions. When given a function and some input arguments it calculates the value of the function. If we have a problem to solve or a question to be answered, we need to bring into this form: function and input values.

Where do we get the inputs? That is the raw data we have in the beginning. Bits and pieces of information which we want to process to get some meaningful results. Certain data items that represent our question.

Where do we get the functions to call? There are four ways to get those.

1. **The built-in functions are already there.** The programming language comes with numerous general purpose functions. We can just start using them.

2. **Some data structures behave like functions.** Vectors are functions from the set of valid indices to the corresponding content. Hash-maps are functions from keys to values. Hash-sets are functions from elements to truthy values.

3. **Higher order functions can create new functions.** Some functions produce functions as their output values. Function composition is the prime example, but all point-free style functions belong to this category.

4. **Writing new functions from scratch.** We can define functions directly by specifying its arguments and its body, which is a bunch of function calls. Thus we still combine existing functions, but in a more manual way.

Functions can be found not just in the core language, but in external *libraries*. These are collections of useful functions, usually for solving some particular type of problems. In the 21st century, software development is more about finding the right library, than programming.

Calling only built-in functions, we can use the language as a calculator, and we are *users*. In case the desired functionality is not available, we need take the role of the *programmer*, and create new functions.

Bibliography

[1] H. Abelson, G.J. Sussman, and J. Sussman. *Structure and Interpretation of Computer Programs*. Electrical engineering and computer science series. MIT Press, 1996. https://mitpress.mit.edu/sicp/, https://sicpebook.wordpress.com/, https://github.com/sarabander/sicp-pdf.

[2] Douglas Adams. *The Hitchhiker's Guide to the Galaxy*. Pan Books, 1979.

[3] Rich Hickey. A history of Clojure. *Proc. ACM Program. Lang.*, 4(HOPL), jun 2020.

[4] J. Roger Hindley and Jonathan P. Seldin. *Lambda-Calculus and Combinators: An Introduction*. Cambridge University Press, New York, NY, USA, 2 edition, 2008.

[5] D.R. Hofstadter. *Gödel, Escher, Bach. Anniversary Edition: An Eternal Golden Braid*. Basic Books. Basic Books, 1999.

[6] Donald E. Knuth. *The Art of Computer Programming, Volume 1 (3rd Ed.): Fundamental Algorithms*. Addison Wesley Longman Publishing Co., Inc., Redwood City, CA, USA, 1997.

Index

*, 12
+, 12
-, 12
/, 12
<, 15
<=, 16
=, 15

and, 43, 45
apply, 32, 72

boolean, 46

capitalize, 18
char?, 17
comp, 11, 65
concat, 22
cond, 42
conj, 24, 55
cons, 20, 55
contains?, 53
count, 21, 60

dec, 10
def, 25
defn, 29
destructure, 63
doseq, 70
drop, 35, 59
drop-while, 35

empty?, 21
ends-with?, 18

false, 44
false?, 44
filter, 39, 45, 53
first, 20
float?, 16
fn, 29
fn?, 16
for, 67

get, 51, 52

hash-set, 53

identity, 9
if, 41, 44
inc, 10
integer?, 16
into, 47
iterate, 58

juxt, 65

keys, 51

last, 22
let, 27, 29
list, 20
loop, 61
lower-case, 18

map, 37, 39, 72
mapv, 72
max, 33, 41, 48
min, 33

neg?, 15
nil, 21, 44
nil?, 21
not, 43
nth, 24
number?, 16

or, 43, 45

partial, 65
pos?, 15
println, 69

rand, 69
range, 34
recur, 61
reduce, 47
reductions, 47
remove, 40
replace, 18
rest, 20
reverse, 22

seq, 54
set, 53
sorted-set, 53
starts-with?, 18
str, 18, 33, 37
string?, 17, 26
symbol?, 26

take, 35, 59
take-while, 35
true?, 44

upper-case, 18

vals, 51
vec, 23
vector, 23

zero?, 15
zipmap, 51